

ITMO-2026-Spring-03, BST

Statement
is not
available
in
English
language

A.1. Простейшее BST

0.3 секунд[⌚], 512 мегабайт

В этой задаче вам нужно написать простейшее BST по явному ключу и отвечать им на запросы:

- $+$ x – добавить в дерево x (если x уже есть, ничего не делать).
- $>$ x – вернуть минимальный элемент больше x или 0, если таких нет.

Входные данные

В каждой строке содержится один запрос.

Все x целые от 1 до 10^9 , количество запросов от 1 до 300 000.

Гарантируется, что все x выбраны равномерным распределением.

Выходные данные

Для каждого запроса вида « $>$ x » выведите в отдельной строке ответ.

входные данные
+ 1 + 3 + 3 > 1 > 2 > 3 + 2 > 1
выходные данные
3 3 0 2

Случайные данные! Не нужно ничего специально балансировать.

В этой задаче нужно написать своё дерево, уже готовыми пользоваться нельзя.

Statement
is not
available
in
English
language

B.1. Простейший полуявный ключ

0.5 секунд[⌚], 512 мегабайт

В этой задаче вам нужно написать BST по **явному** ключу и отвечать им на запросы:

- $+$ x – добавить в дерево x (если x уже есть, ничего не делать).
- $? k$ – вернуть k -й по возрастанию элемент.

Входные данные

В каждой строке содержится один запрос.

Все x целые от 1 до 10^9 , количество запросов от 1 до 300 000.

Гарантируется, что все x выбраны равномерным распределением.

В запросах « $? k$ » число k от 1 до количества элементов в дереве.

Выходные данные

Для каждого запроса вида « $? k$ » выведите в отдельной строке ответ.

входные данные
+ 1 + 4 + 3 + 3 ? 1 ? 2 ? 3 + 2 ? 3
выходные данные
1 3 4 3

Случайные данные! Не нужно ничего специально балансировать.

Вам нужно в каждой вершине поддерживать размер поддерева.

В этой задаче вам нужно написать BST по **неявному** ключу.

Statement
is not
available
in
English
language

C.2. Дебаг-вывод дерева

1 секунда[⌚], 512 мегабайт

Дана последовательность добавлений в дерево поиска (BST). После каждого добавления выводите LxR обход дерева (вывести левое поддерево, вывести ключ, вывести правое поддерево).

Входные данные

Последовательность из n ($1 \leq n \leq 1\,000$) целых чисел от 0 до 10^6 , которые следует добавить. Добавлять нужно именно в таком порядке. Гарантируется, что все числа **различны**.

Выходные данные

После каждого добавления выведите дерево в особом формате (см. примеры). Деревья для удобства чтения разделяйте строкой с одним дефисом. Обратите внимание, число пробелов перед x равно $2 \cdot$ глубина вершины.

Входные данные

```
1
5
9
3
```

Выходные данные

```
x=1 size=1
-
x=1 size=2
  x=5 size=1
-
x=1 size=3
  x=5 size=2
    x=9 size=1
-
x=1 size=4
  x=3 size=1
  x=5 size=3
    x=9 size=1
```

Возможно, вам пригодится:

```
printf("%*sx=%d", 10, "", x);
```

на плюсах и:

```
print(f'{' '*10}x={x}')
```

на питоне.

Statement
is not
available
in
English
language

D.2. Двоичное дерево поиска

1 секунда🕒, 512 мегабайт

Реализуйте сбалансированное двоичное дерево поиска.

Или воспользуйтесь любой стандартной структурой из C++: STL.

Или попытайтесь записать квадрат.

Входные данные

Входной файл содержит описание одной или нескольких операций с деревом.

Операций не больше 10^5 . Все числа целые от -10^5 до 10^9 .

В каждой строке находится одна из следующих операций:

- `insert  $x$` — добавить в дерево ключ x . Если ключ x в дереве уже есть, то ничего делать не надо.
- `delete  $x$` — удалить из дерева ключ x . Если ключа x в дереве нет, то ничего делать не надо.
- `exists  $x$` — если ключ x есть в дереве, «true», иначе «false».
- `next  $x$` — минимальный элемент в дереве, $> x$, или «none», если такого нет.
- `prev  $x$` — максимальный элемент в дереве, $< x$, или «none», если такого нет.

Выходные данные

Выведите последовательно результат выполнения всех операций `exists`, `next`, `prev`.

Следуйте формату выходного файла из примера.

Входные данные

```
insert 2
insert 5
insert 3
exists 2
exists 4
next 4
prev 4
delete 5
next 4
prev 4
```

Выходные данные

```
true
false
5
3
none
3
```

```
while (!seekEof()) { ... }
```

Заходят `set`, умный квадрат, сбалансированное `bst`.

Statement
is not
available
in
English
language

E.2. К-ый максимум

0.5 секунд🕒, 512 мегабайт

Напишите программу, реализующую структуру данных, позволяющую добавлять и удалять элементы, а также находить k -й максимум.

Входные данные

Первая строка входного файла содержит натуральное число n — количество команд ($n \leq 100\,000$). Последующие n строк содержат по одной команде каждая. Команда записывается в виде двух чисел c_i и k_i — тип и аргумент команды соответственно ($|k_i| \leq 10^9$).

Поддерживаемые команды:

- $+1$ (или просто 1): Добавить элемент с ключом k_i .
- 0 : Найти и вывести k_i -й максимум.
- -1 : Удалить элемент с ключом k_i .

Гарантируется, что в процессе работы в структуре не требуется хранить элементы с равными ключами или удалять несуществующие элементы. Также гарантируется, что при запросе k_i -го максимума он существует.

Выходные данные

Для каждой команды нулевого типа в выходной файл должна быть выведена строка, содержащая единственное число — k_i -й максимум.

Входные данные

```
11
+1 5
+1 3
+1 7
0 1
0 2
0 3
-1 5
+1 10
0 1
0 2
0 3
```

Выходные данные

```
7
5
3
10
7
3
```

Напишите своё BST.

Можно любое: avl, treap, rb, splay, что больше нравится. На парах вы проходили avl.

Statement
is not
available
in
English
language

F.2. И снова сумма...

2 секунды⁹, 512 мегабайт

Реализуйте структуру данных, которая поддерживает множество S целых чисел, с которым разрешается производить следующие операции:

- $add(i)$ — добавить в множество S число i (если он там уже есть, то множество не меняется);
- $sum(l, r)$ — вывести сумму всех элементов x из S , которые удовлетворяют неравенству $l \leq x \leq r$.

Входные данные

Исходно множество S пусто. Первая строка входного файла содержит n — количество операций ($1 \leq n \leq 300\,000$). Следующие n строк содержат операции. Каждая операция имеет вид либо «+ i », либо «? l r ». Операция «? l r » задает запрос $sum(l, r)$.

Если операция «+ i » идет во входном файле в начале или после другой операции «+», то она задает операцию $add(i)$. Если же она идет после запроса «?», и результат этого запроса был y , то выполняется операция $add((i + y) \bmod 10^9)$.

Во всех запросах и операциях добавления параметры лежат в интервале от 0 до 10^9 .

Выходные данные

Для каждого запроса выведите одно число — ответ на запрос.

Входные данные

```
6
+ 1
+ 3
+ 3
? 2 4
+ 1
? 2 4
```

Выходные данные

```
3
7
```

Достаточно взять уже написанное BST и добавить к нему нужную функцию.

Запросы так странно генерятся, чтобы заставить вас обрабатывать их в online.

Statement
is not
available
in
English
language

G.3. Перестановки strike back

6 секунд⁹, 512 мегабайт

Вася выписал на доске в каком-то порядке все числа от 1 по N , каждое число ровно по одному разу. Иногда он стирает какое-то число и записывает на его место другое. Количество чисел, выписанных Васей, оказалось довольно большим, поэтому Вася не может окинуть взглядом все числа. Однако ему надо всё-таки представлять эту последовательность, поэтому он написал программу, которая в любой момент отвечает на вопрос — сколько среди чисел, стоящих на позициях с x по y , по величине лежат в интервале от k до l . Сделайте то же самое.

Входные данные

В первой строке лежит два натуральных числа — $1 \leq N \leq 100\,000$ — количество чисел, которые выписал Вася и $1 \leq M \leq 100\,000$ — суммарное количество вопросов и изменений, сделанных Васей. Во второй строке дано N чисел — последовательность чисел, выписанных Васей. Далее в M строках находятся описания вопросов. Каждый запрос на изменение числа в некоторой позиции начинается со слова SET и имеет вид SET a b ($1 \leq a \leq N$, $1 \leq b \leq N$). Это означает, что Вася изменил число, записанное в позиции a на число b . Каждый Васин вопрос начинается со слова GET и имеет вид GET x y k l ($1 \leq x \leq y \leq N$, $1 \leq k \leq l \leq N$).

Выходные данные

Для каждого Васиного вопроса выведите единственное число — ответ на Васин вопрос.

Входные данные

```
4 4
1 2 3 4
GET 1 2 2 3
GET 1 3 1 3
SET 1 4
GET 1 3 1 3
```

Выходные данные

```
1
3
2
```

Это не тривиальная задача про ДО + BST. Есть решение за $\mathcal{O}(\log^2 n)$.

[Codeforces](#) (c) Copyright 2010-2026 Mike Mirzayanov
The only programming contests Web 2.0 platform